



University of
**Southern
Queensland**

CSC6200 Advanced ICT Professional Project

Project INSIGHT.AI

Project Report

Version No.: 4

Document ID: 1

04 May 2026

Team Name: P.D.F Agents

Team Members: Hector Daniel Chavez Orozco (u1168447)

Paulina Casillas Arroyo (u1168446)

Aswathy Sreekala Soman (u1169763)

Faisal Khan Mohammad (u1164395)

Supervisor: Zhaohui Tang

Contents

1 Executive Summary.....	3
2 Methodology Report.....	4
2.1 Methodology Statement.....	4
2.2 Justification.....	4
2.3 Discussion.....	6
3. Technology Report.....	6
3.1 Technology Statement.....	7
3.2 Justifications.....	9
3.3 Discussions.....	14
4 Project report.....	15
4.1 Project Outcome.....	15
4.2 Cost of the Project.....	22
4.3 Discussion.....	26
5 Team management report.....	27
5.1 Team Organisation.....	27
5.2 Team Structure and Roles.....	28
5.3 Team Management.....	29
5.4 Other Discussions.....	32
6 Communication Report.....	32
6.1 Communication and Meetings.....	32
6.2 Other Discussions.....	36
7 Code of Conduct and Ethics.....	36
7.1 Code of Conduct.....	37
7.2 Professional Ethics.....	38
7.3 Privacy.....	40
8 Contribution Distribution.....	42
9 Conclusions.....	43
References.....	44
Appendix.....	45
A1 Meeting Minutes.....	45
A2 Log Sheets.....	52
A3 Video presentation.....	52

1 Executive Summary

Project INSIGHT.AI was commissioned by AKA Studio to resolve the issue of not utilizing locally stored documents. The client needed a solution to retrieve, summarize, and extract intelligence from internal files, which would operate entirely locally so that the data privacy risks would not be involved as in other solutions .P.D.F. Agents developed the project over an eight-week period from 9 March to 8 May 2026, using a Scrum-lite methodology structured around three sprints.

The final outcome is a locally hosted AI-powered document assistant built with Retrieval-Augmented Generation (RAG) architecture. The system allows the user to point at a folder of PDFs, Word, or Excel files, it indexes the contents and can query them using natural language questions in a chat interface. All processing runs completely locally on the user's computer with no need of specific characteristics. At no stage does any document content, query, or response ever get sent out to the cloud. The Llama 3.2 3B model, the BAAI/bge-small-en-v1.5 embedding model, and Streamlit have free open-source implementations. The entire system uses Python, LlamaIndex, Ollama with the mentioned models and Streamlit.

The project team successfully completed the work within eight weeks. The project completely fulfilled all core client requirements. Additional characteristics were added during project execution due to a client's request such as document indexing features, demonstrating the team's responsibility to attend clients requests under an Agile methodology. The actual labor cost was \$25,925 AUD, which had a significant reduction of \$3,042.50 from the original estimate of \$28,967.50 for a total of 342 hours. The variance was mainly due to the removal of the dedicated AI Specialist role and the addition of two Technical Writer roles that were not in the original estimate.

The project has a key technical finding that a lightweight 3B parameter model running on consumer-grade hardware can be relied upon for document retrieval and summarisation for this use case with query response times in the range of 5-15 seconds. The main identified limitation in the model is performance drop on complex and multi-document queries. Also, documenting the inability to handle scanned PDFs or legacy .xls files for future work. Due to hardware issues, the LLM switch during the execution is an important issue that is faced in the work.

The main recommendations for future implementations would be to replace the Streamlit frontend to FastAPI and React so that the UI does not freeze during indexing, upgrade the embedding model for better retrieval accuracy on complex queries, and implement incremental re-indexing to detect and re-index the changed files.

2 Methodology Report

2.1 Methodology Statement

Project INSIGHT.AI was developed following a Scrum-lite methodology, which is a variant of the Agile methodology. This methodology was chosen as it adapted well to the project's characteristics and constraints. As the adoption of this project introduced new challenges and uncertainty for all team members, it was highly necessary to choose a methodology that allowed the team to work in a flexible environment, allowing for continuous changes to requirements and definitions, and ensured constant communication with stakeholders throughout the development process.

2.2 Justification

The project was developed under a limited time and resources environment; therefore, the team adopted a hybrid Agile methodology, Scrum-lite. This allowed to develop the project in complete alignment with the same core principles of Scrum methodology, transparency, inspection, and adaptation, without the necessity of the rigid ceremonies at the beginning of each sprint and formally defined roles, which were impractical given the situation where most team members had to participate in multiple tasks due to team's limited size and the project's duration. One of the key benefits of this methodology introduced to the project was the work structure based on sprints, which allowed for more efficient use of the resources and strategic communication with the clients, who were constantly updated on progress and advances.

At the beginning of each sprint, all team members met to identify the tasks to be completed during that period and assign ownership of each task to a team member. A task was considered complete once it had been implemented, tested, and reviewed by at least one other team member. This was applied across all sprints, to ensure quality and traceability of deliverables.

The Scrum-lite methodology had several positive impacts on the evolution of the project, which reflected on the communication within team members and with external participants such as stakeholders, as well as the overall project performance. The key positives observed were the following:

- Flexibility to change or reprioritize requirements based on new findings.
- Early delivery of working components allowed LLM integration to be tested iteratively, allowing them to identify potential issues before final implementation.
- Transparent progress tracking via sprint goal.
- Regular sprint reviews ensured that the project was always aligned with stakeholder expectations.
- Space for dedicated time for research and technical investigation during implementation.

- It allowed for structural changes to the project without the need of discarding prior work or restarting the process.

Along with this adoption also came some challenges and limitations, mostly related to sprint planning and project management. The main observed drawbacks were the following:

- Scope management was challenging with a small multi-role team.
- Difficult to estimate cost and timeline, as scope and structure can change over time.

Structure

The project was structured around sprints, with the different phases broken down across three sprint iterations. At the beginning of the project, in the original project plan it was stated that the project would be completed in four sprints of approximately three weeks each. However, during execution it was found that two of those sprints were referring to the same interdependent objectives which needed to be completed simultaneously to develop a meaningful deliverable at the end of the sprint. Considering them as two separate sprints was introducing unnecessary time and reducing efficiency. Then, it was decided to consolidate those two sprints into a single extended sprint, to better reflect the actual flow of work.

The final sprint structure consisted of three sprints, and was the following:

Sprint	Start date	End date	Duration	Phase
Sprint 1	09/03/2026	23/03/2026	2 Weeks	Initiating, Planning
Sprint 2	23/03/2026	20/04/2026	4 weeks	Executing
Sprint 3	20/04/2026	08/05/2026	2 weeks	Monitoring, Controlling, and Closing

As previously mentioned, the Executing phase was initially divided into two separate sprints. However, during execution it was noted that waiting until completing one sprint to start with the other was delaying some activities that were dependent, and this would cause reduced efficiency. Consequently, Sprints 2 and 3 were merged into a single sprint, enabling better time management and allowing the team to allocate additional effort to the testing and debugging process within Sprint 2. The total project duration remained eight calendar weeks, running from 09/03/2026 to 08/05/2026/.

It is also important to highlight that even if activities were organized within sprint boundaries, team members were not strictly restricted to only working on the current sprint's tasks. At points where capacity was available, team members were able to begin preparatory work on activities from the following sprint, provided those activities had no dependency on incomplete current-sprint deliverables.

2.3 Discussion

The first sprint was focused on project initiation and planning. The sprint goal was to establish and set all the foundations for the project, defining methodologies, requirements, and technical architecture of the LLM. This goal was fully met, and the final result was the development of a complete project plan by the end of the sprint.

The second sprint covered all activities related to the execution phase, considering both backend and frontend integration. This was the longest sprint out of all three, reflecting the complexity of the activities in this and the consolidation of the original two execution sprints. A key deviation from the original plan occurred during this sprint, the selected model was to be changed due to complexity and computational limitations, not all team members were able to access the tool due to computational capabilities the decision of this change was based on the scope and characteristics of the project were it was stated that the program ought to be able to run on basic computational systems. By sticking to the methodology's principles, we were able to attack the integration issue without compromising the overall sprint deliverable.

Lastly, Sprint 3 covered monitoring, controlling, and project closure. The sprint goal was to complete final validation and produce all required project documentation and deliverables. The completion of this sprint marks the end of the project and the handling of the final result to the client.

Overall, the Scrum-lite methodology worked well for this project. The sprint-based structure allowed us to better identify deliverables, break them into manageable activities, and track progress effectively across the timeline. Thanks to this structure the team was able to achieve all milestones on time, and also handle all technical risks, restructuring, and changes that were necessary during the project's life. The flexibility of the methodology allowed for all of this without the necessity of restructuring the framework and therefore employing extra time, correctly responding to change, and improving the outcome delivery.

In future approaches, the team would adopt Scrum-lite again for a project of similar scope and complexity. However, some changes would be made to future iterations. First, a stricter definition of done would be established at the beginning of the project rather than being redefined during the project. Second, sprint scope management would be planned more, and allocating extra time for technical uncertainty where outcomes are less predictable.

3. Technology Report

3.1 Technology Statement

At its core, INSIGHT.AI is a local document assistant. Users point it at a folder of files and then ask questions about those documents through a chat interface. Everything runs on the user's own machine. No data leaves the computer, no cloud accounts are needed, and there are no API costs. The table below summarises the technologies used and what each one does in the project.

Technology	What It Does
Python 3.13	The programming language used for the entire application, covering both the interface and the backend logic.
Streamlit	Turns the Python script into a web app. It provides the chat window, sidebar controls, index management panel, and session handling without any front-end code.
Ollama	Runs AI models locally. It downloads the model, loads it into memory, and handles requests from the app. Think of it as a local AI server.
LLaMA 3.2 (3B)	The language model that reads retrieved document chunks and generates answers. At 3 billion parameters, it is small enough to run on a standard laptop.
LlamaIndex	The retrieval framework. It handles text chunking, embedding, vector storage, index persistence, and retrieval when a question is asked.
bge-small-en-v1.5	The embedding model from BAAI, loaded via HuggingFace. It converts text into numerical vectors so the system can measure similarity between a question and stored chunks.
LlamaIndex VectorStore	LlamaIndex's built-in vector store. It holds the document embeddings and persists them to disk in the ~/INSIGHT_AI_storage folder, so indexes survive between sessions.
python-docx	A Python library used to extract text from Word (.docx) files by reading each paragraph.
pypdf	A Python library used to extract text from PDF files, reading each page individually.
openpyxl	A Python library used to read Excel spreadsheets (.xlsx/.xls), extracting data from each sheet and row.
VS Code	The code editor used by the team for writing, debugging, and testing all Python code throughout the project.

GitHub	Used for version control and collaboration. The team stored source code in a shared repository and managed contributions through branches and pull requests.
--------	--

The workflow has two phases. First, the user types a folder path into the sidebar and clicks “Index Documents.” The app reads every supported file in that folder, extracts the text using the appropriate library (python-docx for Word files, pypdf for PDFs, openpyxl for spreadsheets), splits the text into smaller chunks, converts each chunk into a vector using the embedding model, and saves the whole index to disk. A JSON registry keeps track of which folders have been indexed and what documents they contain. The second phase is querying. When the user types a question, the system converts it into a vector, compares it against the stored chunks, pulls out the five closest matches, and passes them to LLaMA 3.2. The model reads through the retrieved context and writes an answer. A conversation memory buffer keeps track of earlier messages so follow-up questions work naturally.

Users can manage multiple indexes from the sidebar. They can create new ones, switch between existing ones, view the documents in each index, and delete indexes they no longer need. All of this is stored locally on the user’s machine.

System Requirements

Since everything runs locally, the user’s hardware matters. The application needs to load both a language model and an embedding model into memory at the same time, which sets the floor for what kind of machine can run it.

Component	Minimum	Recommended
Operating System	Windows 10 / macOS 12 / Ubuntu 20.04	Windows 11 / macOS 14 / Ubuntu 22.04
RAM	8 GB	16 GB or more
Storage	10 GB free (models + index data)	20 GB+ free for larger model options
GPU	Not required (runs on CPU)	Dedicated GPU with 4 GB+ VRAM
Python	Python 3.10+	Python 3.13
Ollama	Latest stable release	Latest stable release
Internet	Required for initial model download	Required for initial model download

The application was developed and tested on a standard laptop without a dedicated GPU. Response times were acceptable for most queries, though indexing large document folders and loading the embedding model for the first time took noticeably longer on CPU.

3.2 Justifications

This section walks through why we picked each technology, and what we found worked well or did not work so well once we started building with it.

Streamlit

We went with Streamlit because it meant we could build the entire interface in Python. No HTML, no CSS, no JavaScript. For a project where the real work is happening in the backend, that trade-off made sense. It has built-in chat components, session state management, and sidebar widgets, which cover everything we need for index management and the chat interface. The downside showed up when indexing large folders: because Streamlit runs on a single thread, the whole interface freezes until the indexing finishes. There is also not much flexibility if you want a more custom layout.

Strengths	Limitations
Zero front-end code required	UI freezes during long indexing operations (single-threaded)
Chat UI, sidebar, and session state come built in	Limited control over layout and styling
Any change to the script updates the interface instantly	Not ideal if the app ever needs to scale to production use

Ollama

Ollama was the obvious pick for local model serving. You install it, run one command to pull a model, and it handles everything from there: loading the model, managing GPU or CPU resources, and serving requests. No API keys, no internet connection needed during use. The catch is that performance depends entirely on whatever hardware the user has. On a machine without a dedicated GPU, responses can feel slow, and there is not much we can do about that from the application side.

Strengths	Limitations
One command to download and start a model	Response speed is entirely hardware-dependent

Handles GPU and CPU resource management on its own	Not much visibility into how it allocates memory
No cloud dependency, no API costs at all	Slower on machines without a GPU

LLaMA 3.2 (3B)

We needed a model that could run on a normal laptop and still give reasonable answers. LLaMA 3.2 at 3 billion parameters fit that requirement. It handles most document questions well enough, especially when the answer sits clearly in one of the retrieved chunks. Where it falls short is on questions that need the model to piece together information from several different parts of a document, or when the retrieved text is ambiguous. It also occasionally makes things up, which is a known limitation of smaller models.

Strengths	Limitations
Runs on consumer hardware, no high-end GPU needed	Struggles when the answer spans multiple chunks
Gives coherent answers for most document queries	Sometimes produces inaccurate or made-up details
Small enough to keep response times reasonable	Smaller context window limits how much text it can consider

LlamaIndex

LlamaIndex was chosen because it ties the whole retrieval pipeline together: chunking, embedding, storing, persisting to disk, and querying. We did not use its built-in document readers since we needed more control over how each file type was parsed, but its indexing, storage, and retrieval components saved us a lot of work. The built-in persistence was especially useful. Calling `storage_context.persist()` saves the entire index to a local folder, and `load_index_from_storage()` brings it back, so users only need to index a folder once. The trade-off is that when retrieval results are poor, the framework's abstractions make it harder to figure out where the problem is.

Strengths	Limitations
Handles the full indexing and retrieval pipeline	Abstractions make debugging retrieval issues tricky
Built-in persistence saves indexes to disk	Hard to tell which component causes poor results

Integrates directly with Ollama and HuggingFace models	Documentation varies in quality between versions
--	--

bge-small-en-v1.5 (HuggingFace)

We chose BAAI's bge-small-en-v1.5 as the embedding model because it is lightweight, performs well on retrieval benchmarks for its size, and runs locally through HuggingFace's integration with LlamaIndex. It downloads automatically the first time the app runs and does not need Ollama or any separate service to operate. The downside is that it only supports English, and being a smaller model, it is less accurate than larger embedding models on nuanced or complex queries. Indexing on CPU is also noticeably slower with large document sets.

Strengths	Limitations
Lightweight and fast for a model of its accuracy	English only, no multilingual support
Downloads automatically via HuggingFace, no extra setup	Less accurate than larger models on complex queries
Runs entirely locally with no external API calls	First-time download and loading can be slow on CPU

LlamaIndex Built-in Vector Store

Rather than adding an external vector database like ChromaDB or FAISS, we used LlamaIndex's built-in vector store. It stores all the embeddings in memory during use and can persist them to disk as JSON files. This kept the setup simple. Users do not need to install or run a separate database. The index files are saved under ~/INSIGHT_AI_storage, and each indexed folder gets its own subdirectory. The limitation is that it loads the entire index into memory, which could become a problem with very large document collections. It also does not support concurrent access from multiple users.

Strengths	Limitations
No external database to install or manage	Loads entire index into memory, not ideal for huge collections
Persistence to disk means indexes survive between sessions	No concurrent multi-user access
Integrated directly into LlamaIndex, zero extra config	Less query flexibility than a dedicated vector database

Custom Document Loaders (python-docx, pypdf, openpyxl)

We wrote our own document loading logic instead of relying on LlamaIndex's built-in readers. This gave us full control over how each file type was handled. python-docx reads Word files paragraph by paragraph. pypdf extracts text page by page from PDFs. openpyxl reads Excel spreadsheets sheet by sheet and row by row, joining cell values with pipe separators. Each document's text is also prepended with its filename, which helps the model cite sources in its answers. The downside is that these are basic text extractors. python-docx ignores tables and images inside Word files, pypdf struggles with scanned or image-heavy PDFs, and openpyxl loses all cell formatting and formulas.

Strengths	Limitations
Full control over how each format is parsed	python-docx ignores tables and images in Word files
Filename metadata included for source citation	pypdf cannot handle scanned or image-heavy PDFs
Three file types supported: .docx, .pdf, .xlsx/.xls	openpyxl loses formatting, formulas, and merged cells

Visual Studio Code

VS Code was the team's code editor throughout the project. We chose it because everyone was already familiar with it, and it has strong Python support through extensions like Pylance and the Python debugger. The integrated terminal made it easy to run the Streamlit app and Ollama side by side during development. The Git integration was also useful for committing changes and managing branches without leaving the editor.

Strengths	Limitations
Team already had experience with it	Can feel heavy with many extensions installed
Excellent Python extension support (Pylance, debugger, linting)	Occasional lag when working with very large files
Built-in Git integration and terminal	

GitHub

GitHub was used for version control and team collaboration. The source code, README, and documentation all lived in a shared repository. Each team member worked on their own branch and merged changes through pull requests, which helped avoid conflicts and kept the main

branch stable. GitHub also served as the final deliverable location, since the project required a working prototype with source code available in a repository.

Chatbot Pipeline

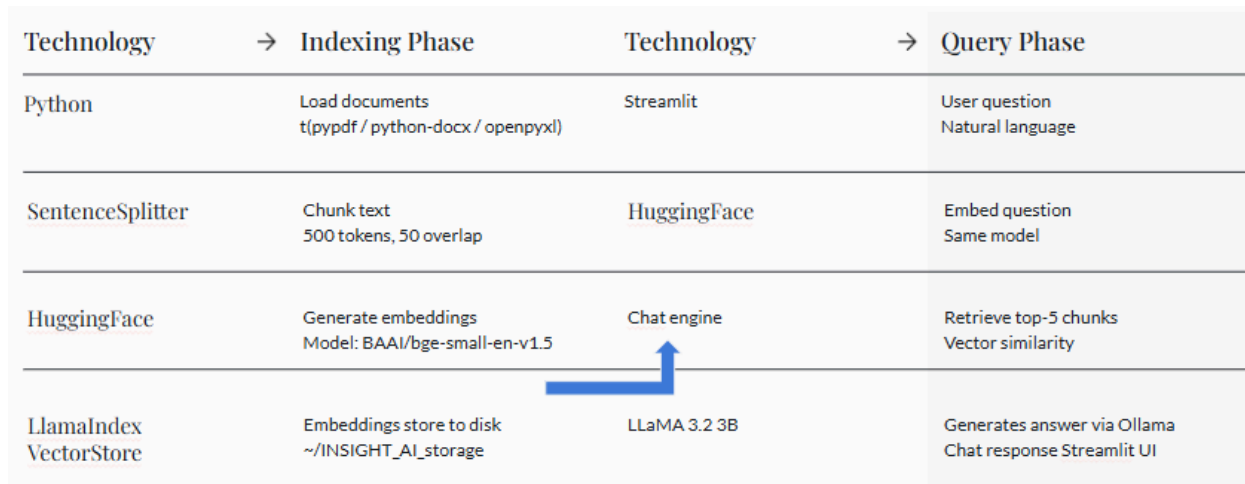


Image 1. Chatbot Pipeline

Indexing Process (Left Side)

- The diagram consists of two sides of the application process.
- The left one stands for indexing - this is the process that takes place during a click on "Index Documents" by the user.
- First, the application loads the content of each file in the chosen directory through the appropriate loader:
 - pypdf for PDF files
 - python-docx for .doc files
 - openpyxl for .xls/xlsx files
- Then, the text of the documents loaded is divided into 500 tokens with a 50-token overlap.
- After that, each tokenized text is embedded in the bge-small-en-v1.5 model.
- The created index is stored in ~/INSIGHT_AI_storage for future use since the process of indexing is done only once.

Query Handling Process (Right Side)

- The right side of the diagram describes the process of a query handling.
- If a user asks the system any question, the text will be converted into a vector through the same embedding model as was used for indexing.

- Afterward, the system retrieves the top five closest vectors based on similarity from the previously mentioned persisted vector store.
- The system sends both the five vectors and the initial text to LLaMA 3.2.
- The latter reads everything and composes an answer, which appears in the chat window of Streamlit.

3.3 Discussions

Future Considerations

If the team were to build something similar again, there are several things we would do differently.

- Front-end framework: Streamlit got us a working interface quickly, but the UI freezing during indexing was a constant frustration. Next time, we would consider splitting the application into a FastAPI backend and a React front-end. That way, indexing could happen in the background while the user continues chatting. It would take more work upfront, but the result would feel much more polished.
- Language model: The 3B model was good enough for simple, single-chunk questions. But for anything requiring the model to connect dots across multiple sections of a document, the answers were hit or miss. A 7B or 8B model would likely do better here. Since Ollama makes it easy to swap models, this would be a one-line change in the code.
- Document extraction: Our custom loaders handle basic text extraction, but they miss a lot. Tables in Word documents, images in PDFs, and formulas in spreadsheets are all lost. For PDFs specifically, a library like `pymupdf4llm` would preserve table structures by converting to Markdown. For Word files, a more capable parser could extract tables alongside paragraphs. These improvements would make a noticeable difference in answer quality for documents with complex formatting.
- Embedding model: `bge-small-en-v1.5` is fast and convenient, but it only supports English and is less accurate than larger alternatives. We would consider upgrading to `bge-base` or `bge-large` for better retrieval accuracy, or switching to a multilingual model if the application needed to handle documents in other languages. Running embeddings on GPU would also help with indexing speed.
- Vector storage: `LlamaIndex`'s built-in store works well for single-user use with moderate-sized document collections. But it loads everything into memory, which would not scale to thousands of documents. A future version could use a dedicated vector database like `ChromaDB`, `Qdrant`, or `LanceDB` for more efficient storage and retrieval.
- Change detection: Right now, if a user modifies a document in an indexed folder, the app has no way to know. They have to delete the index and rebuild it from scratch. A future version could track file modification times and sizes, then only re-index the files that have actually changed.

4 Project report

4.1 Project Outcome

Problem Definition Review

The main and initial purpose of Project INSIGHT.AI was to help AKA Studio solve its need to be able to access all of the underutilized local documents in a more efficient way, being able to easily summarize their contents and obtain significant insights that similar public Large Language Models would offer, but still maintaining security and minimizing the risk of possible data leaks that often come with the use of these. This problem definition was established in the project plan and remained as the guideline throughout the entire development process. The final outcome of the project, the local LLM, directly addresses this problem and offers a solution for it, adapting to the previously defined scope, offering as a final result a local Large Language Model that offers an AI chatbot where users can query and receive specific information from locally stored files.

Detailed Project Outcome Description

Non-technical description

The final outcome of the project is an AI-powered prototype of a conversational chatbot that allows users to extract, query, and summarize information from a single or a large collection of local documents without the necessity of manually reviewing each individual file, a document assistant. The primary use case is information retrieval from personal or organizational document repositories stored on a local machine.

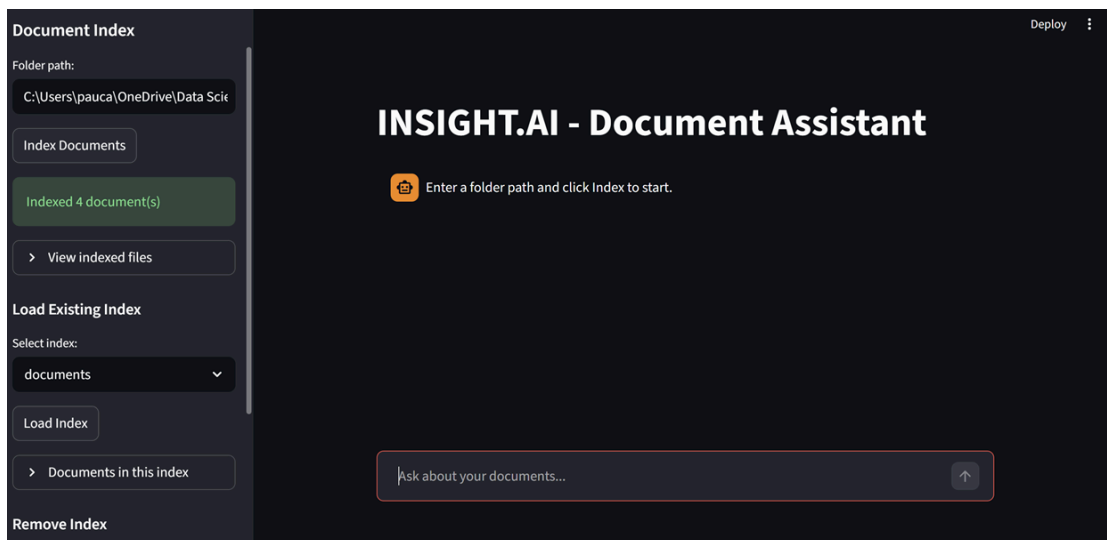


Image 2. [INSIGHT AI](#) interface

The user interaction follows this workflow: first the user needs to provide the system a local folder path, which contains the documents to be analysed, PDF, Word, or Excel files, then the system indexes all files within that directory, and the user can then submit natural language queries such as requesting summaries of a specific document, asking a specific question about its contents, or retrieving particular information that spans across multiple files. Then the system processes the query against the indexed document set and returns an accurate response obtained from the relevant documents.

The system also allows the user the opportunity to manage the indexed documents. Once the system recognizes the path, it shows all the documents stored in this index, so the user can see which documents they can retrieve information from.

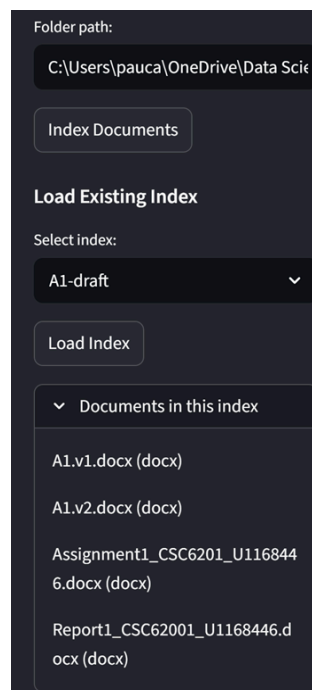


Image 3. Document Index menu

Additionally, the user can add multiple indexes and move between them, so the system can easily identify which documents the user is referring to.

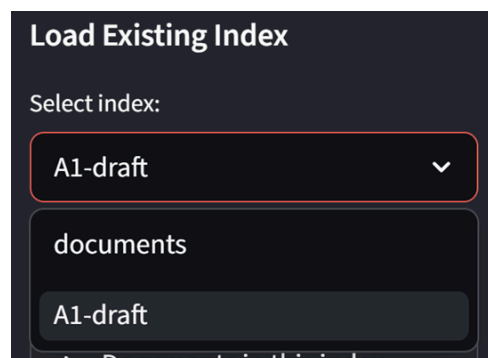


Image 4. Document Index menu

The user can also manage the system indexes, with the option to remove selected indexes so the system no longer sees them, and no more documents are retrieved from them.

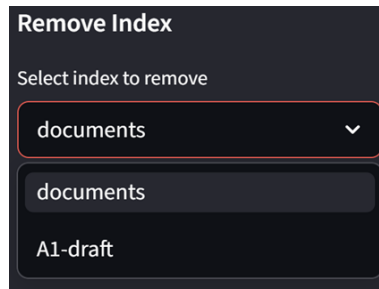


Image 5. Document Index menu

All these features and characteristics offer the user full control over the documents that the system can access and manage according to their needs..

Technical description

The system is built on a Retrieval-Augmented Generation (RAG) architecture, which combines a document retrieval pipeline with a locally hosted LLM to generate accurate responses grounded strictly in the provided documents. The following describes each core component of the system.

Document Ingestion and Processing

The document ingestion process reads and extracts text content from supported file formats, including PDF, Word and Excel. PDF text extraction is handled by the *pypdf* library, Word files are processed using *python-docx*, and Excel files are read using *openpyxl* with the *data_only* flag to ensure bringing the results from formulas rather than raw expressions. To improve model identification of document sources, the filename is attached to the extracted text of each document before processing. Scanned PDFs and xls Excel 97-2003 files are not currently supported as a known limitation in the prototype.

Text Chunking

Once extracted, document text is split into smaller segments using the *LlamaIndex SentenceSplitter* function, configured with a chunk size of 500 tokens and an overlap of 50 tokens. This overlap ensures that contextual continuity is maintained across chunk boundaries, reducing the risk of losing relevant information.

Embedding Generation

Each text chunk is converted into a high-dimensional vector representation using the *BAAI/bge-small-en-v1.5* embedding model, accessed locally through the *HuggingFace* library.

These vector embeddings capture semantic meaning of each chunk, enabling similarity-based retrieval that goes beyond simple keyword matching. This embedding model runs locally, ensuring that no document content is transmitted externally during the process.

Vector Index and Storage

The generated embeddings are locally stored in a LlamaIndex *VectorStoreIndex*. By having these vectors stored locally on the user's computer, it means that documents do not need to be reprocessed on subsequent sessions, the index is loaded directly from disk. For persistence, a JSON-based registry file is maintained alongside the indexes to track which folders have been indexed and which documents they contain.

Query Engine and Language Model

User queries are processed through LlamaIndex chat engine, configured with a *similarity_top_k* value of 5, meaning the five most semantically relevant chunks are retrieved and passed to the language model as context. The language model used is Llama 3.2 3b, accessed locally through the Ollama framework. Ollama manages the local deployment of the model without requiring an internet connection. The engine maintains a conversation memory buffer of 3,900 tokens, allowing users to ask follow-up questions within the same session. A strictly defined system prompt instructs the model to answer only from the provided document context, cite source documents in its responses and explicitly indicate when requested information cannot be found, mitigating the risk of hallucination.

Frontend Interface

The user interface is built using Streamlit and it is structured with a sidebar for index management, including folder indexing, index selection, document visibility, and index deletion, and a main chat area where users can submit natural language queries and receive responses. The dashboard runs locally on and is launched via batch script, requiring no technical knowledge from the end user beyond the initial setup.

On the collaboration side, we set up a GitHub repository early on and agreed on a branching workflow. Each team member created feature branches for their work and submitted pull requests before merging into the main branch. VS Code's built-in Git tools made this process fairly smooth for day-to-day commits, and we used GitHub's interface for reviewing pull requests and resolving any merge conflicts.

All components of the system operate within the local machine environment. No document content, query or response is transmitted to any external service at any stage of the pipeline. All the models used run locally on <http://localhost:8501/>, and all indexes and registry files are stored on the user's local file system. This architecture directly satisfies the primary security requirement established in the project plan and differentiates the system from cloud-based AI alternatives.

Quality review

The final prototype successfully addresses the main problem and meets the client requirements, providing a functional locally hosted AI chatbot prototype capable of querying and summarizing document information. The system correctly supported the different file formats and demonstrated reliability during internal testing and client demonstrations. The main limitations identified primarily relate to the model's capability limitations inherent to running a lightweight LLM on standard consumer hardware, which was one of the infrastructure specifications from the client.

User perspective

The system was constantly evaluated internally by the development team and demonstrated to the client during the fortnightly meetings. But no other external user testing was conducted within the project timeline. Therefore, the presented quality assessment is based on the intended user perspective, and informed by internal testing observations and client feedback.

Feature / Characteristic	Assessment
Usability	The interaction with the chatbot proved to be intuitive, allowing the user for straightforward use without requiring complex technical guidance. In the different meetings with the client when showing the use-flow of the system they didn't highlight or mention any specific changes for the setup. One of the identified improvement areas was the folder path input and the index section, which can be more automatic instead of requiring the user to manually handle them.
Reliability	During internal testing and client demonstrations, the system proved to retrieve relevant information from indexed documents in response to user queries about broad or specific content. No critical failures were observed during client demonstrations. Identified limitations during internal testing include the inability to process scanned PDFs, as the system relies on text extraction rather than optical character recognition. Queries that are highly ambiguous or reference information not present in the indexed documents are handled by the system prompt, which instructs the model to explicitly indicate when the requested information cannot be found rather than generating an unsupported response.

Completeness	All main client requirements were successfully addressed. The document indexing functionality was introduced in a project in direct response to a client request and was successfully implemented within the existing sprint structure. The final system delivers folder-based document indexing, PDF, Word and Excel file ingestion, natural language querying, document summarization, specific information retrieval, multi-index management, and index deletion — covering the full scope of agreed requirements. Excel file support was also added beyond the original scope as a further improvement to the system's utility. However, there were some out-of-scope features that were discussed in meetings with the client that due to time limitations were not possible to execute, such as implementing a comparison tool for different document versions or work with other types of documents such as power point. These can be held in a future project.
--------------	---

Technical perspective

Feature / Characteristic	Assessment
Maintainability	The system follows modular architecture with clear separation between the three main layers: the Streamlit frontend, the Python/LlamaIndex backend, and the Ollama LLM inference layer. Each component can be updated or replaced independently, for better performance. The model can be swapped, without requiring changes to the indexing or retrieval logic managed by LlamaIndex. Similarly, the embedding model can be replaced without modifying the frontend or query engine. The RAG pipeline itself is structured in distinct stages, meaning each stage can be modified or optimised independently. The JSON-based registry file that tracks indexed folders is also decoupled from the vector index, making it maintainable without touching the core retrieval logic. The batch file further isolates environment setup from application logic, simplifying the startup and onboarding process.

Reliability	The system demonstrated reliable results during internal testing. Two file processing limitations were identified: scanned PDFs without an embedded text layer are not supported as the system relies on text extraction rather than optical character recognition, and legacy .xls Excel files are also unsupported. Error handling ensures that unsupported or unreadable files are skipped during indexing without causing a system failure, with the affected file logged accordingly. Users are informed through the interface when a file cannot be processed. Additionally, the strictly defined system prompt significantly improves response reliability by instructing the model to answer only from provided document context and explicitly indicate when the requested information cannot be found, directly mitigating the hallucination risk inherent in LLM-based systems. This is particularly critical for a document intelligence tool used in a professional business context. These limitations are documented in the setup guide and represent targeted areas for improvement in future iterations.
Efficiency	System performance is hardware dependent as all inference runs locally via Ollama rather than through a cloud service. The Llama 3.2 3B model was selected specifically for its ability to run on standard consumer-grade laptops without a dedicated GPU, which was a constraint established by the client given the absence of defined infrastructure specifications. Query response times during internal testing averaged approximately 5 to 15 seconds per query, with variation dependent on query complexity and the number of retrieved chunks (configured at top-k of 5). The choice of the BAAI/bge-small-en-v1.5 embedding model balances semantic retrieval quality with local inference speed, making it appropriate for consumer hardware. Initial document indexing time scales with the size of the document collection, with small to medium sets completing within a few minutes. Subsequent sessions load the pre-built vector index directly from disk, eliminating reprocessing time entirely. The conversation memory buffer of 3,900 tokens allows users to ask follow-up questions within the same session without performance degradation.

Portability	The system has a defined set of environment dependencies: Python 3.x, LlamaIndex, Streamlit, pypdf, python-docx, openpyxl, and HuggingFace libraries on the backend, and Ollama with the Llama 3.2 3B model installed locally. A batch file is provided to automate environment setup and application launch, reducing configuration effort on a new machine. However, portability is constrained by the local-first architecture: Ollama must be installed separately, and the model must be downloaded before first use, which represents a non-trivial setup step for non-technical users. Deployment to a different machine requires replicating the Python environment, installing Ollama, and downloading the model. This local-first constraint is a deliberate trade-off that directly satisfies the client's primary security requirement — no document content, query, or response is transmitted to any external service at any stage of the pipeline. A setup guide was produced by the Project Analysts to support deployment on new machines.
-------------	---

4.2 Cost of the Project

Since this project was developed for academic purposes, no real costs were incurred during its realization. Also, as this was meant to be a first approach, primarily a prototype, of how a project like this would work, the client wanted something that didn't represent any cost to them; therefore, all the proposed technologies were open-source tools.

However, an actual cost analysis that reflects the effort, time, and resources invested by the team was conducted, time and resources invested by the team. All costs were calculated using the hourly rates and cost structure defined in the Project Plan. This provides a basis for comparing estimated costs against the actual values.

Technologies cost

As previously mentioned, all the technologies and tools used in the project were open source, meaning that they do not charge for use and no cost was incurred for technologies.

Labour cost

When the project's actual development began, some changes were made to the team's structure, as outlined in the project plan. According to each team member's capacities and time limitations, it was decided that only two developers were needed, one backend and one frontend, mainly since the team realized that it was not necessary that all four members were coding at the same time. However, one more team role was defined which was Technical Writer, two team members were assigned to this. Their primary tasks were to execute all activities related to project's documentation, setup guides, architecture documents, user guides, comparative analysis and all

required documentation for ensuring that every change and decision made in the model's architecture was well documented and supported, as well as testing of the different versions of the model.

Additionally, these two members also participated in all teams' decisions regarding changes and new technologies suggestions to improve team functioning and time management. It was decided to keep the structure that way to ensure that all tasks were completed on time and met all requirements. Therefore, the final team structure was the following.

- Team lead
- Frontend developer
- Backend developer
- Technical writer (x2)

Considering this, a new cost was introduced to the initially estimated costs. The final estimates were based on current Australian market standards for junior professionals in each role.

Role / Position	Hourly Rate
Project Lead	\$90 AUD
Backend Developer	\$85 AUD
Frontend Developer	\$75 AUD
Technical Writer / Analyst	\$70 AUD

Labor costs were calculated based on these rates and considering the actual logged hours for each team member across all sprints and the project's duration, as recorded in the Activity Log Sheets available in this document's Appendix. All durations are expressed in calendar hours.

Role	Team member	Hourly Rate	Hours logged	Total Cost
Project Lead	Hector Daniel Chavez Orozco	\$90.00	15 hrs	\$1,350
Backend developer	Hector Daniel Chavez Orozco	\$85.00	85 hrs	\$7,225
Frontend developer	Faisal Khan Mohammad	\$75.00	82 hrs	\$6,150
Technical Writer	Aswathy Sreekala Soman	\$70.00	80 hrs	\$5,600
Technical Writer	Paulina Casillas	\$70.00	80 hrs	\$5,600
Total			342 hrs	\$25,925

The total hours spent in the project were 342 hours including the work of all team members, these hours reflect the total hours dedicated to the project. These include all development and documentation activities, client meetings, and everything related to the project. As observed the team member with the less hours was the Project Lead, this doesn't mean that there was not a proper team management during the project but mainly the project lead tasks were mainly executed while communicating with the client and having meetings, besides that the project's organization tasks were executed by all team members without having a specific leader role within the team organization just for external purposes.

Besides that, the hours spent on the project were roughly equal for all team members, with the developers spending a little more time than the others, reflecting the project's focus on the technological side and the need for more time for development tasks.

Therefore, the total cost of the project was \$25,925 AUD, which reflects only the labor cost, as that is the only actual cost incurred. As previously mentioned, there was no actual cost for the selected technologies for the development. This final cost deviate from the original estimation, and an actual comparison is shown in the following section.

Estimated vs Actual Cost Comparison

Cost Component	AI Estimate	Actual Cost	Deviation
Software	\$0.00	\$0.00	\$0.00
Project Lead	\$6,750.00	\$1,350.00	-\$5,400.00
Backend developer	\$7,687.50	\$7,225.00	-\$462.50
Frontend developer	\$6,205.00	\$6,150.00	-\$55.00
AI Specialist	\$8,325.00	\$0.00	-\$8,325.00
Technical Writer	\$0.00	\$5,600	+\$5,600.00
Technical Writer	\$0.00	\$5,600	+\$5,600.00
Total	\$28,967.5	\$25,925.00	-\$3,042.50

The main differences between the estimated and actual costs were in labor costs, the developer roles did not show a considerable difference. The frontend developer, which was labeled as Software developer in the project plan, had only a small difference of \$55 AUD, mainly caused by differences in work hours, same with backend developer, which was previously named simply as Developer. Then the AI specialist role defined in the project plan was not carried forward into the final team structure. Upon commencing development, it became clear that the associated responsibilities, mainly model selection, evaluation, and integration decisions, were most effectively handled collectively by the full team rather than assigned to a single dedicated role. As a result, the AI Specialist allocation of \$8,325.00 was eliminated from the actual cost.

However, one other roles was defined: technical writer. This role was considered necessary because a lot of documentation was required throughout the project, and because many other tasks that were not taken or specifically assigned to any other team member were assigned to these team members. A more detailed list of task ownership by each of team members is shown below.

Work hours by team member

The following table maps the approximate time that each team member spent on each sprint and phase of the project. A more detailed task ownership can be found in the activity log sheets in the Appendix.

Phase / Sprint	Team member	Sprint	Hours Logged	Status
Initiating	Hector Daniel Chavez Orozco	1	~ 6 hrs	Complete
Initiating	Faisal Khan Mohammad	1	~ 6 hrs	Complete
Initiating	Aswathy Sreekala Soman	1	~ 10 hrs	Complete
Initiating	Paulina Casillas Arroyo	1	~ 10 hrs	Complete
Planning	Hector Daniel Chavez Orozco	1	~ 8 hrs	Complete
Planning	Faisal Khan Mohammad	1	~8 hrs	Complete
Planning	Aswathy Sreekala Soman	1	~ 20 hrs	Complete
Planning	Paulina Casillas Arroyo	1	~ 20 hrs	Complete
Executing	Hector Daniel Chavez Orozco	2	~ 50 hrs	Complete
Executing	Faisal Khan Mohammad	2	~ 50 hrs	Complete
Executing	Aswathy Sreekala Soman	2	~ 30 hrs	Complete
Executing	Paulina Casillas Arroyo	2	~ 30 hrs	Complete
Monitoring and controlling	Hector Daniel Chavez Orozco	3	~ 15 hrs	Complete
Monitoring and controlling	Faisal Khan Mohammad	3	~ 15 hrs	Complete
Monitoring and controlling	Aswathy Sreekala Soman	3	~ 8 hrs	Complete

Monitoring and controlling	Paulina Casillas Arroyo	3	~ 8 hrs	Complete
Closing	Hector Daniel Chavez Orozco	3	~ 6 hrs	In progress
Closing	Faisal Khan Mohammad	3	~ 3 hrs	In progress
Closing	Aswathy Sreekala Soman	3	~ 12 hrs	In progress
Closing	Paulina Casillas Arroyo	3	~ 12 hrs	In progress

This table summarizes roughly the amount of time that each member spent in the different phases of each sprint, these times include the different tasks related to the project and that involve all communication with the client and team meetings. As a result of the defined roles, Aswathy and Paulina dedicated more time to the non-technical phases, which required more documentation, and on the other hand, Faisal and Daniel dedicated more time to the technical activities, which were more focused on the development of the model's prototype.

4.3 Discussion

The project managed to deliver on its primary objectives which allows the user to efficiently retrieve and summarize the information contained in local documents through an entirely local AI model. The final outcome effectively meets the client's data privacy requirements by ensuring that processing only takes place on the user's machine without any data transmission.

Based on performance, the system proved to be reliable on standard queries and document summarization tasks. Nonetheless, using a lightweight 3B parameters model created constraints when handling more complex questions requiring synthesis of information from various documents. There's an optimization tradeoff at play where you can run it on consumer hardware.

Additionally, the project was completed at a lower cost that anticipated because there was no longer a dedicated AI Specialist, therefore, the cost associated with this role was dismissed. Although this cost reduced the total cost but had challenges in estimating accurate resource allocation in adaptive Agile-based projects.

The prototype not only fulfills all the essential functional requirements but also lays a strong foundation for enhancement in the future in spite of these limitations. The project as a whole demonstrates the feasibility of a local RAG-based document assistant, while also highlighting some of the key trade-offs in cost, performance and capability of the system.

5 Team management report

This section offers an overview of the P.D.F Agents team's approach to organising, managing and structuring collaborative work during the various stages of Project INSIGHT.AI. It considers how the team was structured and how they communicated, the technical and non-technical roles of the team members, the challenges they encountered during the three sprint cycles and the approaches they took to successfully deliver a prototype within the eight-week project timeframe. The insights presented in this section are based on the team's recorded sprint reviews, meeting minutes and the commit history on GitHub, which together form a verifiable account of team actions from the project's inception to handover.

5.1 Team Organisation

At the beginning of the project, the team agreed to be self-organising with the Scrum-lite model chosen for the project. Rather than being directed from above, each team member had clear primary responsibilities in line with the four main technical areas of the new system, but was also able to contribute as needed across roles in the event of workload or technical constraints. The choice of this self-organisation structure was made to accommodate the high-speed iterative development process, in which hierarchical team structures would have only added to the burden of coordination in a four-person team over a short timeframe.

In practice, the team organised its coordination across two time scales. On the inside, weekly meetings of the team offered a structured forum for review of the sprint, allocation of tasks, and identification of risk factors. In between, the team coordinated their activities via a dedicated Microsoft Teams channel, set up at the start of Sprint 1, which was used for general updates, sharing artefacts, and ad-hoc decision-making. The team's GitHub repository stored all project resources (source code, draft documentation, version history), giving everyone in the team access to the latest version of the project at any time, regardless of when or where they worked.

Externally, fortnightly meetings with our client (Charles Lee, Lavinia Adams) and academic supervisor (Linda) served as external checkpoints. The team coordinated all official communication with these stakeholders, and took feedback from these meetings in the form of meeting minutes, which were shared with the entire team via Teams within 48 hours. This double loop iteration (weekly internal review, fortnightly external review) guaranteed that the team's sprint progress was evaluated relative to the team's plan as well as the client's and supervisor's expectations.

A key organisational change was needed during execution, as the original four-sprint plan was adjusted to three sprints. The Executing phase had originally been divided across two sprints; but during Sprint 2, it became apparent that backend pipeline and LLM integration were so tightly coupled that there was little meaningful work that could be delivered at the boundary between them. Combining these into an extended Sprint 2 (23 Mar - 20 April 2026) provided a more

accurate representation of the workstream and enabled the team to use the extra time to focus on extensive testing and debugging. The project timeline was still eight calendar weeks and ended on 8 May 2026, and this change was communicated and agreed upon by the client and the supervisor.

5.2 Team Structure and Roles

The team settled on a role-based model, where each team member was responsible for the main technical element of the project. This was based on the technical competence of each team member, and also their contribution to the academic deliverables of the project. The roles and responsibilities are outlined in the table below:

Team Member	Role	Primary Responsibilities
Hector Daniel Chavez Orozco	Backend	Timeline oversight, sprint coordination, client and supervisor communication, GitHub repository management
Paulina Casillas Arroyo	Technical Writer	Preparation, structuring and documentation of the Final Project Report and distribution of meeting minutes throughout the project .
Aswathy Sreekala Soman	Technical Writer	Preparation, structuring and documentation of the Final Project Report.
Faisal Khan Mohammad	Frontend	Streamlit dashboard design and implementation, user interface, frontend-backend integration and frontend development

In addition to these technical roles, each team member also had an informal role contributing to the project's cohesiveness. Hector was the main liaison with external stakeholders, including the client, and ensured feedback was correctly captured and turned into sprint items. Paulina and Aswathy handled document formatting, version management of the report drafts and the submission process, and tracked the risk register during the sprints. Faisal drove all matters related to the usability of the system, including the scripted demonstrations of the prototype prepared for each client meeting, and ensured that the Streamlit prototype was always at its best for the client's review.

In reality, these roles were (intentionally) blurred. The Scrum-lite process that the project adopted was based on the expectation that a small four-member team, working under tight time constraints, would need to be able to flexibly redistribute work within the team. This was critical

during Sprint 2, when the model substitution created additional work for the all team members. Instead of this hindering the team, it expanded documentation work - writing sections of the technical architecture document - to other team members to keep sprint productivity high while ensuring the quality of the AI integration work was maintained. This ability is considered by the team as a key organisational benefit of this project.

5.3 Team Management

5.3.1 Sprint process and definition of done

The project adopted a three-sprint Scrum-lite process, which came about as a result of an iterative planning process. Each sprint started with a meeting to discuss the backlog, clarify ownership of the tasks and define the sprint goal. This sprint goal was used to prioritise all tasks for that sprint -if a task did not align with the sprint goal, it was postponed until the following sprint's backlog rather than being included in the current sprint's workload.

A task was defined as done when it had been implemented, tested and reviewed by another team member. This definition of done was defined early in the development cycle and consistently applied to all three sprints, particularly during Sprint 3 where the final validation, documentation and delivery of the solution were the main deliverables. The need for a peer review prior to task completion was formalised as a pull request on GitHub, which provided an audit trail of review activity. For tasks that failed peer review, the assignee was provided with feedback and the task was not merged until all issues had been identified for further work. This ensured that the master branch of the repository was a reviewed and working version of the prototype.

In line with the Scrum-lite approach, team members were not constrained to working only on the tasks assigned for the current sprint. Providing they had time and the task was free of dependencies on any current-sprint products, they could start work on next-sprint tasks. This mitigated delays that naturally occur between sprints, and was particularly helpful between Sprint 1 and Sprint 2 where the developers could begin preliminary work on the Ollama environment while the rest of the team was finishing the final project planning documentation.

The three sprints played out as follows:

Sprint	Start	End	Duration	Primary Focus and Deliverable
Sprint 1	09/03/2026	23/03/2026	2 weeks	Initiating and Planning -stakeholder identification, requirements gathering, system

				architecture design, Project Plan deliverable
Sprint 2	23/03/2026	20/04/2026	4 weeks	Executing -data ingestion pipeline, text chunking, embedding generation, duplicate detection, LLM integration (including model substitution), Streamlit dashboard, backend-frontend integration; first functional prototype
Sprint 3	20/04/2026	08/05/2026	2 weeks	Monitoring, Controlling, and Closing -integration testing, performance testing, security testing, bug fixing, feedback integration, all documentation, client handover

5.3.2 Challenges and Responses

A number of major issues were encountered during the course of the project that tested the team's ability to respond without undermining the quality of the final product. Here are some of the more significant challenges faced in the three sprints.

The most significant technical challenge was switching the chosen LLM model in Sprint 2. The model identified in the Project Plan was not able to run consistently on all of the team's hardware as the hardware of typical student laptops did not have the computational capabilities. Given the project's requirement was that the prototype run on this type of hardware -as stated in the scope and non-functional requirements -a model that could not be run reliably on the development team's hardware was unviable. Technical Writers conducted the research and, adhering to the Scrum-lite manifesto of responding to change, found a lighter model that was acceptable for local inference and summarisation quality for the project, and did not exceed the hardware specifications. The switch was made without redoing work on the pipeline, the new model was checked as open-source and permissively licensed before being integrated into the pipeline and the client and supervisor were notified at the fortnightly meeting. The sprint was delivered on time, in extended Sprint 2.

Another issue was managing the sprint scope in a small, cross-functional team. With the team sharing multiple responsibilities, the team's reaction to the unplanned work required to substitute the model just at the time when the developer was also faced with the challenge of navigating the complexities of integrating the product's back-end and front-end. The team responded to this by having a mid-sprint meeting -outside the scheduled weekly meetings -to re-prioritise the backlog

and assign documentation work to team members with spare capacity. This ad hoc, yet structured, approach averted a sprint failure resulting from the model substitution.

Another issue was the difficulty of accurately estimating costs and timelines with an adaptive approach. The initial four-sprint plan was revised to account for the consolidation of the execution phase and when individual tasks took longer than expected in Sprint 2. The team coped with this by leaving room in the plan for Sprint 3 -as suggested in the Project Plan's risk mitigation plan - and by moving Excel and PowerPoint assistance to the stretch goal category.

5.3.3 Lessons to be Taken Forward

The following approaches were most successful and should be considered for future team projects of a similar nature:

- Combining sprints when necessary: Consolidating the two Executing sprints into a longer Sprint 2 was a good thing to do. Continuing with an artificial sprint boundary during development would have led to an additional changeover cost and lower productivity during the technically most challenging period of the project. Teams should be encouraged to adjust sprint boundaries mid-project as the flow of work between inter-dependent tasks dictates, if the change is communicated to and agreed with the stakeholders.
- Mid-sprint check-ins during high-complexity sprints: Scheduling an extra (informal) team meeting to re-prioritise the backlog when the model substitution introduced unexpected work was a good way of not losing momentum in the sprint until the planned weekly check-in. During high-complexity sprints working on uncertain technical tasks, proactive scheduling of a mid-sprint check-in would be an improvement.
- Walk-through of the prototype: Scripting a walk-through of the prototype rather than a live demo reduced technical risks during client demos. This is an easy practice that should be normalised at the beginning of any project that includes client demonstrations.
- GitHub for version control and communication: When GitHub was used not just as a version control system, but also for communication (in the form of comments on pull requests and code reviews), the context of technical decisions was maintained. This eliminated the need to rely on verbal commitments from previous meetings, and made it easier for members to interpret and build on code in the integration-intensive Sprint 2.
- Open sprint reviews prior to meetings: A team-only sprint review 48 hours before each fortnightly client or supervisor meeting meant that the team was well-versed and consistent in its feedback. This was especially important in the meeting in which we announced the model change, as it allowed us to agree to the wording of the change and how it should be presented.

5.4 Other Discussions

The balanced 25% contribution in the Project Plan was largely upheld for the duration of the project. The model change in Sprint 2 resulted in an extra workload at a certain point; this was accommodated by replacing the documentation responsibilities with other team members (and not ignoring them). The team's adherence to the code of conduct principle of transparent contribution reporting ensured that this was discussed in the weekly team meeting and documented in the team meeting minutes.

A team lesson from this project relates to the definition of done. The team knew that the definition of done -implemented, tested, and peer-reviewed -was consistent, but learned and refined it as the project progressed. During Sprint 1, some tasks were defined as complete once the implementation and self-review were complete; from Sprint 2, the peer review was more strictly enforced. Having a detailed written definition of done for the project at the project kick-off meeting, and applying it to the first task, would aid in quality traceability from the first deliverable and is the team's major recommendation for future projects of a similar nature.

Finally, the team observed that the agility of the Scrum-lite approach, while advantageous during project execution, at times made it harder to convey a specific feeling for project progress to stakeholders outside the team who were accustomed to progress measured by waterfall-style milestones. Creating a basic visual progress report (such as a sprint burndown chart) for each of the fortnightly client meetings would have alleviated this communication issue and given the client a better view of the project's progress relative to the plan in the moment.

6 Communication Report

In the Project Plan, communication was recognised as one of the key aspects for the success of Project INSIGHT.AI, as it was critical to ensure that all team members were on the same page with a client from industry, and supervisor from academia, over an eight-week development period. This section offers a detailed analysis of the communication practices at the team, client and supervisor level, its support and the issues encountered and lessons learnt, to be taken into future projects. Meeting minutes referred to in this section can be found in Appendix .

6.1 Communication and Meetings

6.1.1 Internal Communication Structure

Internal communication took place via a dedicated Microsoft Teams channel, which was set up at the start of Sprint 1. The channel was the main coordination workspace for instant updates, file sharing, blocker raised and "soft" decision-making between the weekly meetings. Members were required to read and reply to the channel within 24 hours, which was upheld throughout the project. The absence of conversation history in Teams meant that anyone who missed a period of

time could return and see all decision making and discussions without having to ask another member for information -which was a particularly important feature in the technically-complicated Sprint 2 when team members were developing different aspects of the system in parallel.

Weekly team meetings were held regularly during the three sprints. These meetings had a pre-determined agenda, sent by the Project Lead at least 24 hours in advance. The agenda typically included: the team's progress towards tasks in the current sprint plan; blockers, risks or changes in scope that the team needed to discuss; allocation of tasks to team members and confirmation of ownership for the week ahead; and the preparation of items for client or supervisor meetings. This agenda ensured meetings stayed on-topic and on-time. A rotating scribe documented the meetings and circulated the minutes via Teams within 24 hours of the meeting, providing a record of decisions made and tasks allocated.

In addition to the regular weekly meetings, an unscheduled meeting was called in the middle of Sprint 2 to address re-prioritisation of the team's backlog as a result of unforeseen work brought about by the LLM model substitution. This spontaneous meeting was convened by the Project Lead and attended by all team members; it took about 45 minutes, and produced a document of task assignments which was shared via Teams and noted in the subsequent week's minutes. The team appreciated this agile approach -holding additional meetings as needed rather than waiting for the regular scheduled cycle -as a strategy to deal with unforeseen circumstances without jeopardising the sprint.

6.1.2 Communication with Client and Supervisor

The client and supervisor were present at fortnightly meetings, which offered external feedback and validation. The Project Lead facilitated these meetings, which were attended by the entire team and external stakeholders were provided with an agenda 24 hours prior to the meeting. The agenda for each client and supervisor meeting -sprint overview, prototype demonstration, risk and issues update, and open feedback -meant that stakeholders were presented with an accurate update at every meeting.

The team prepared and practiced demonstrations of the prototype for each client meeting. This practice was introduced after the first client demonstration, in which a live walkthrough of an early Streamlit prototype unveiled a minor bug in the product's user interface. Subsequent client meetings used a scripted demonstration that only tested the components known to be stable, and documented unstable or in-progress features with verbal commentary. This removed any technical surprises for client demos and is highly recommended for the future.

The minutes of each external meeting captured all feedback, and were circulated to the whole team by the Project Lead within 48 hours. Any changes to the prototype functionality or project scope as a result of client or supervisor feedback were documented within the sprint backlog and

implemented accordingly, ensuring that client and supervisor feedback was tracked from request to completion. A summary of the project's external meetings is shown in the table below:

Date	Type	Participants	Key Outcomes
10 March 2026	Client meeting	Full team + Charles Lee	Client requirements validated; technology stack confirmed; project scope and prototype expectations agreed
23 March 2026	Supervisor meeting	Full team + Dr Tang	Sprint 1 deliverable (Project Plan) reviewed and approved; Sprint 2 objectives and success criteria confirmed
30 March 2026	Client meeting	Full team + Charles Lavinia Adams	WBS to be updated continuously; GitHub repository to be shared with AKA Studio; weekly progress reports with WBS snapshot to be maintained; data ingestion, preparation and retrieval pipeline to continue (due 13 April)
10 April 2026	Client meeting	Full team + Charles Lee, Lavinia Adams	WBS to be updated continuously; GitHub repository to be shared with AKA Studio; weekly progress reports with WBS snapshot to be maintained; data ingestion, preparation and retrieval pipeline to continue (due 13 April)
25 April 2026	Client meeting	Full team + Charles Lee	Current model features are enough, the team has to make sure that everything runs correctly and prepare documentation for handover. Comparison feature was deemed out of scope and will not be pursued due to time and scope constraints.

6.1.3 Internal Communication

The internal communication of the project was professional and effective. The Teams communication channel was kept up to date by the team and the 24-hour standard response time was upheld. Significant decisions (such as the model swap, sprint merge, and documentation task reallocation in Sprint 2) were made collectively, rather than being imposed by one party, which preserved team ownership and control of the project's technical direction, despite technical challenges.

The GitHub channel was a great addition to the communication channels. Comments on pull requests and code reviews captured the reasoning for technical choices in the context of the code being discussed. This allowed members to find out why a particular implementation was chosen - for instance, why a particular chunking configuration was used - in the comments of the review thread, rather than having to rely on memory of past conversations or meetings. This approach dramatically limited the number of clarifying questions members had to ask each other and is highly recommended for future projects of the software development team.

6.1.4 Communication Challenges Encountered

The most significant challenge to communication in the project was the lack of synchronicity between the sprint and fortnightly client meetings. The fortnightly schedule was set at the beginning of the project and could not be easily changed; however, as sprint length and deliverable dates changed (particularly after the sprint consolidation), the meeting dates often did not align at the end of sprints. In one case during Sprint 2, the team had to present work to the client in mid-sprint as opposed to at the end of a deliverable. This required a subtler presentation of the progress made, since the team had to make it clear what components were complete, what components were being worked on and what components had yet to begin work. To make this clear, the team prepared a progress report, which was sent to the client prior to the meeting, listing the state of each component. This strategy worked well and avoided misinterpretations of the state of the prototype at that stage.

The second communication issue was the volume of information to be communicated during Sprint 2, in which the model substitution, sprint consolidation, and the integration of the backend and frontend were happening in parallel. During this period the amount of updates being circulated in the Teams channel was the greatest that the team had experienced and on a couple of occasions updates required follow-up questions because they did not establish the necessary context in the update itself. This was resolved by the team using a more structured message format for technical updates for the rest of Sprint 2 -a brief summary of the context, the update, and what was needed from others -that eliminated the need for follow-up questions.

6.1.5 Meeting Effectiveness and Proposed Improvements

The weekly meetings were an effective use of time, with the prior distribution of the agenda format keeping discussions focused and avoiding extended problem solving. In Sprint 2, however, the technical challenges of integrating the LLM into the application and the need to substitute the model led to some of the weekly meetings exceeding the planned hour of discussion time, with problem-solving discussions intermingled with review-and-planning discussions. The team would resolve this issue in future projects by creating separate technical working sessions (to focus on debugging and problem solving) in addition to the weekly review and planning meeting. The review-and-planning meeting could then be limited to an hour while still identifying that a technical working session was required (to be scheduled accordingly).

The supervisor and client meetings were well conducted. The formalised approach used for the second client meeting and beyond was much better than the earlier ad hoc demonstration, and should be the default approach from the beginning in subsequent projects. Another improvement suggested by the team would be to send the client a brief summary of each meeting within 48 hours of the meeting to confirm decisions and actions. Although meeting minutes were circulated to the team, a client summary would provide an extra level of agreement and a point of reference for the client between meetings.

6.2 Other Discussions

A practice the team found useful in terms of communication was the identification of discussion and decision-making in meeting minutes. This meant that items were identified as "information shared", "discussion had", or "decision made", with the decision written out in full, and the name of the person responsible for any action item identified. This approach facilitated determining whether decisions had been implemented between meetings, and allowed any questions about what had been decided to be resolved. This is a straightforward but very effective form of minute taking, which the team would like to see adopted by all project teams.

The team also found the GitHub repository to be an informal, but effective, meeting preparation tool. Members could review the history of commits and pull requests for the past week to get a sense of what had been accomplished since the last meeting, without having to verbally update everyone on each item. This shortened the review part of the weekly meetings, and increased the time for planning and risk management. Making this practice more formal (asking team members to review the GitHub activity log prior to each week's meeting) would make it an even stronger practice in future projects.

Lastly, the team also observed that the communication process with the client was greatly simplified by the client's level of engagement and feedback. In projects with less engaged clients, or where clients provide more general feedback, a more formalised feedback collection process -asking specific questions during the weekly meetings based on the deliverables of the sprint -would be beneficial to gather feedback for each meeting.

7 Code of Conduct and Ethics

In this section we reflect on the implementation of the professional ethics and code of conduct statements made in the Project Plan during the course of Project INSIGHT.AI. It reviews the situations during the three sprints in which these commitments were engaged, how well they were maintained and how future projects could be improved accordingly. These reflections are framed by the ACS Code of Professional Conduct (v2.1), the ACM Code of Ethics and Professional Conduct and the IEEE-CS/ACM Software Engineering Code of Ethics, which were

included in the ethics and conduct section of the Project Plan and signed by all four team members at the start of the project.

7.1 Code of Conduct

7.1.1 Situations and Issues Encountered

There were a number of particular situations throughout the three sprints that tested out the team's code of conduct promises. Below is an overview of each:

The most significant situation that tested code of conduct was the LLM model swap in Sprint 2. The Project Plan stated a model to be run on Ollama would be the locally run LLM. After it became apparent that the model would not run on all of the team's machines, the team had to decide whether to select a different model for the demonstration. First, the commitment to transparent stating of ability and limits (ACS Ethics, Honesty a and b) required that the team not try to build the prototype with a solution that only worked on some systems. Second, the code of conduct commitment to require verification of the licence of any new tool before it is added to the stack (ACS CoC, Principle 3; IEEE Principle 2.02) required the model's licence to be checked. Third, the commitment to open communication with the client (ACS CoC, Principle 4) required the change to be announced at the next fortnightly meeting rather than made without the client's knowledge. The three commitments were met: the limitation was acknowledged, the new model was checked to be open-source and permissively licensed before being added to the stack, and the client and supervisor were briefed at the next meeting with an explanation of why there was a change, and that the new model still met the functional requirements.

The use of client data was a continuing conduct commitment for all three sprints. AKA Studio supplied representative document samples for development and testing, and the team's code of conduct stipulated that these files be held locally, excluded from GitHub via .gitignore and that no internet-connected AI be used to process any of the files. This was adhered to throughout the project.

The project managed intellectual property considerations. The chosen technology stack -Python, Streamlit, LlamaIndex, Ollama and the final LLM -is all permissively licensed and open source. A list of all libraries used, including name, version and licence information, was kept during the project and included in the technical documentation submitted to AKA Studio at the end of the project. No team member added a new dependency to the project unless its licence was checked and discussed in the team's weekly meeting.

The code of conduct was enforced through structural safeguards in the development process and behavioural norms that were maintained throughout the three sprints.

Structurally, GitHub branch protection ensured all pull requests were peer reviewed before they were merged to the main branch. This translated the code of conduct commitment to peer review

to ensure quality and prevent untested code being merged into the production branch, and provided a record of every review and approval within the project. The constant rule that no AKA Studio documents should go through any AI system connected to the internet was complemented with a structural solution that negated the technical possibility of data sharing, rather than creating a behavioural rule.

At the behavioural level, the team used the format of the weekly status report including an issues section that was required to be written to include risks and blockers. This ensured that the model substitution, the sprint merge, and the period of being behind schedule were disclosed in the team's internal and external status reports instead of being hidden or downplayed. The dispute resolution process (team discussion followed by vote) defined in the Project Plan was in place throughout the project, and the team used this process to decide on the model substitution and sprint consolidation.

7.1.2 Success and Lessons Learned

The code of conduct set out in the Project Plan worked well for the project. The code of conduct framework provided the team with a common set of commitments to make decisions under pressure - most notably, for the model substitution, where the urgency of the decision needed to be weighed against confirming the licence for the replacement model and communicating the change to users. The fact that these commitments were written prior to the incident meant that the team did not need to decide on the ethics of the situation "in real time"; the code of conduct told them what to do, and they did it.

One opportunity to better utilise the code of conduct was in regularly reviewing its commitments at the end of each sprint rather than as a document-specific commitment at the beginning of the project. In retrospect, ethical review of the project only happened in Sprint 1 (when the Project Plan was developed) and Sprint 3 (when this report was created). A short, structured code of conduct review session at each sprint review (five minutes discussing whether any code of conduct issues arose during the sprint and how they were dealt with) would build into the team's sprint cycle as opposed to the end of the project.

7.2 Professional Ethics

7.2.1 Professional Ethics - Issues and Actions

The key dimension of Project INSIGHT.AI was the ethical design and development of an AI-based system which interacts with potential proprietary business documents of a real client under Australian privacy legislation. The system -which processes, analyses, and summarises local documents using a local LLM -imposed on the team ethical considerations that go beyond typical software engineering practice and which involved active choices at various points in the project.

The most important ethical consideration was the commitment to represent the prototype's abilities and deficiencies accurately to the client. This involved two points of interaction throughout the project. First, during the design phase, the Project Plan explicitly documented the prototype's limitations - the approximate size limit of 10MB for the directories, performance variations on student-grade computers, the exclusion of Excel and PowerPoint from the scope of the initial version, and the lack of persistent query history by default - and the plan to disclose them to the client prior to the final demonstration. These constraints were documented in the installation guide provided to AKA Studio at the end of the project. Second, the model substitution in Sprint 2 brought with it a new responsibility: the client had agreed to the project on the basis of a particular technology stack, and switching out a part of the stack without informing the client would have led to a misrepresentation. The team explained the change at the following fortnightly meeting, with reasons and confirmation that the function was met. This complied with ACS Ethics (Honesty a and b) and ACM Principle 1.3, and the client was satisfied with the change.

Responsible use of technology (IEEE Principle 3; ACM Principle 1.2) was implemented through the offline system design and the team's policy to not process client documents using external AI tools. There were no calls to external AI APIs at development, testing or demonstration time. LLM inference was performed locally by Ollama, confirmed by network monitoring tests in Sprint 3's security testing.

Progress was reported truthfully in all three sprints. Progress reports were accurate in terms of task completion and identified risks and blockers. No task was reported as complete until it had been completed according to the definition of done, no risk or issue was left out of a report to improve the reported progress of the sprint. This approach fostered and sustained client and supervisor trust in the project.

7.2.2 Tackling Strategies

The most potent ethical control mechanism was architectural. In setting the system to operate offline (no API calls to an external service; no default persistent query logs; no cloud storage), the major ethical threats of AI systems for document processing were addressed by design, not by policy. This "ethics by design" approach, in line with ACM Principle 1.2 and IEEE Principle 3, is much stronger than relying on behavioural controls, because it prevents the technical possibility of harm rather than merely setting a policy against it.

The policy of regular, fortnightly meetings with the client served an ethical purpose beyond managing the project. It established a periodic opportunity for any changes in the agreed scope, technology and schedule -such as the model swop -to be disclosed transparently and immediately and, for the client to give their informed consent to proceed. This avoided the possibility of the client receiving a service different from the one they contracted, without being given a chance to oppose or request changes.

The code review process for all pull requests addressed an ethical as well as a technical goal. Code review - and the changes that result - can find not just bugs, but also design choices that may have unintended privacy, security or usability consequences. The code review process, which meant a second pair of eyes scrutinised all code changes before they were merged into the main branch of the repository, minimised the chance of any of the changes introducing a privacy vulnerability or an architectural departure from the agreed offline-only system design without the team's collective knowledge.

7.2.3 Lessons for Future Projects

The team reflected that there were a number of ways in which professional ethics practice could be improved:

- Ethics review at each sprint boundary: Reviewing the ethical issues at the end of each sprint (what data was involved in the sprint; what tools were used; whether there were any new ethical issues that arose from the work undertaken in the sprint) would embed ethics considerations as a part of the project process rather than just project-start and project-end processes.
- Recording ethical rationale in technical designs: If a technology design or implementation decision is made for ethical or privacy reasons, that rationale should be recorded in the technical design document, along with the technical rationale. This makes the ethical rationale to future system users (developers or clients) who extend the system and ensures the rationale is not lost when the original system designers are no longer around.
- Ethics review for mid-project technology changes: The model change was well managed, but it pointed out the original ethics and privacy review at project planning was against the original technology stack. If the technology is changed during the project, a re-review of the technology change should be conducted before the change is integrated into the system to ensure the new tool does not introduce any additional risks.

7.3 Privacy

7.3.1 Issues and Actions Involving Privacy

Privacy was the main non-functional requirement that shaped the system's design from the very beginning. The client's key reason for choosing a locally hosted AI document intelligence tool instead of a cloud-based option was to keep business documents under the organization's control and to stop them from being sent to third-party servers. This meant that every technical and operational choice in the project had privacy impacts. Any breach of the offline-only principle would have threatened the project's main value to the client.

The following privacy-related design and operational choices were made during the three sprints:

- Data minimization was implemented as a default behavior from the early architecture discussions. The prototype does not save query history, extracted document content, or vector embeddings between sessions unless the user specifically activates the optional export feature. This design choice follows Australian Privacy Principle 11 (APP 11 - security of personal information) by ensuring that no leftover data from a session remains available after the session ends. This decision was recorded in the system architecture document given to AKA Studio, so any future developer working on the prototype understands that the non-persistence of session data is a deliberate design choice with a privacy purpose, not an oversight.
- The lack of third-party data transmission was confirmed through code review and network monitoring during Sprint 3's security testing phase (WBS 5.1.3). This verification provided the team with documented proof that the privacy guarantee was upheld in the operational prototype, not just in theory.
- AKA Studio's test document files were managed according to the procedures outlined in the Project Plan. Files were kept in a local, access-controlled folder on the development machine designated for testing, added to .gitignore before the repository was set up to prevent accidental inclusion in any commit, and removed from all development machines after completing Sprint 3 testing. No team member copied test files to personal cloud storage, personal devices, or any tool other than the locally hosted prototype.
- The model substitution during Sprint 2 also needed a privacy review before implementation. The new model was evaluated to make sure it operated completely offline via Ollama and did not create new external data transmission paths for instance, no telemetry, no model update checks, and no usage reporting before it was added to the pipeline. This check was conducted jointly by all team members and documented in the sprint's meeting notes.

7.3.2 Effectiveness of Privacy Strategies

The privacy strategies used were effective throughout the project. The offline-only architecture removed the main privacy risk by design. The .gitignore policy and local-only test file storage provided safeguards against exposure in the repository and cloud. The Sprint 3 network monitoring verification offered documented proof that the architectural privacy guarantee was achieved in the operational system. Together, these strategies formed a layered privacy protection model—architectural, procedural, and verification-based—that is much stronger than any single control applied alone.

The choice to verify the privacy implications of the model substitution before integration was a significant example of privacy awareness during execution. It would have been easy to view the model change as simply a technical decision and move forward without a privacy review. The fact that the team took the time to conduct one shows an advanced understanding of privacy as a design requirement, not an afterthought.

7.3.3 Improvements for Future Projects

Based on the team's experience across the three sprints, the following improvements to privacy practices are suggested for future projects involving client data or AI-powered document processing:

- A written data handling checklist at project kickoff: A checklist that clearly specifies where test files may be stored, who may access them, under what conditions they may be copied, and when they must be deleted should be agreed upon and signed by all members during the project's first meeting. This removes any confusion about data handling expectations and ensures everyone starts with a shared understanding of the requirements.
- A privacy step in the definition of done: For any task involving client or user data, the definition of done should include a specific privacy verification step. This step should confirm that no test files were committed to the repository, no new external network calls were created by new dependencies, and no data was unintentionally kept beyond the session. This ensures privacy compliance is checked at task completion rather than only during the final sprint's security testing.
- A sprint-level privacy review: A brief structured review at the end of each sprint should cover what data was handled, where it was stored, whether any unintended copies were made, and whether any new privacy implications arose from the sprint's work. This would identify privacy issues early and stop them from building up until the final testing phase.
- A privacy review for mid-project technology changes: Any technology change introduced after the initial ethics and privacy review should trigger a focused privacy reassessment of the changed component before integration. The model substitution in this project was handled correctly, but the process for doing so was improvised at the moment instead of following a pre-established procedure. Defining this process from the start would make it more efficient and consistent.

8 Contribution Distribution

Student Name	Student ID	Signature	Contribution (%)	Date
--------------	------------	-----------	------------------	------

Hector Daniel Chavez Orozco	u1168447		25%	04/05/2026
Paulina Casillas Arroyo	u1168446		25%	04/05/2026
Faisal Khan	u1164395		25%	04/05/2026
Aswathy Sreekala Soman	u1169763		25%	04/05/2026

9 Conclusions

Project INSIGHT.AI was developed to address a specific challenge that AKA Studio was facing, the inability to efficiently access and extract insights from a huge collection of locally stored documents, without compromising the security and confidentiality of sensitive data. The project has successfully delivered a functional prototype that helps to solve this problem.

Throughout the eight-week development period, the project team designed and implemented a locally deployed AI document analysis system built on a RAG architecture, putting together open-source technologies like Python, LlamaIndex, HuggingFace, Ollama and Streamlit. The system allows users to index local document folders, submit natural language queries and receive accurate responses, all processes entirely on the local machine without transmitting any data to external services.

A Scrum-Lite methodology was followed for the development process, this approach helped with managing effectively the technical complexity of integrating multiple AI components within a short eight-week timeline. Regular client engagement ensured that the prototype evolved in alignment with organisational needs.

The prototype demonstrates that a secure, locally hosted AI tool is feasible to deploy on standard business hardware without specialised infrastructure or licensing costs. While the prototype has documented limitations like the lack of scanned PDF support or constrained multi-document comparison, these represent opportunities for future development rather than fundamental flaws in the proposed approach. The modular architecture of the system was specifically designed to allow individual components, such as language model, embedding

model or vector store, to be upgraded independently as organisational needs and hardware capabilities.

In conclusion, this project has delivered a working prototype that validates the core concept, meets the primary requirements defined at the beginning of the project, and provides AKA Studio with a solid foundation upon which a production system can be built. The project demonstrates that organisations can leverage the capabilities of AI for document analysis while maintaining full control over their data, a balance that is extremely important in today's data-sensitive business environment.

References

- AI, N., 2023. Artificial intelligence risk management framework (AI RMF 1.0). URL: <https://nvlpubs.nist.gov/nistpubs/ai/nist.ai>, pp.100-1.
- Di Palma, G., Scendoni, R., Tambone, V., Alloni, R. and De Micco, F., 2025. Integrating enterprise risk management to address AI-related risks in healthcare: Strategies for effective risk mitigation and implementation. *Journal of Healthcare Risk Management*, 44(4), pp.25-33.
- Frewin, G.D. and Hatton, B.J., 1986. Quality management—procedures and practices. *Software Engineering Journal*, 1(1), pp.29-38.
- Hurzhii, V. and Nizheiko, K., 2025. INTEGRATION OF ARTIFICIAL INTELLIGENCE METHODS INTO PROJECT QUALITY MANAGEMENT: REAL-TIME CONTROL OF COMPLIANCE WITH STANDARDS. *Baltic Journal of Economic Studies*, 11(5), pp.207-213.
- Ispas, L., Mironeasa, C., Severin, T.L., Cerlincă, D.A. and Mironeasa, S., 2025. Artificial Intelligence Applications in Risk Management Within Integrated Management Systems: A Review. *Systems*, 13(11), p.967.
- Mhaskey, S.V., 2024. Integration of artificial intelligence (AI) in enterprise resource planning (ERP) systems: Opportunities, challenges, and implications. *International Journal of Computer Engineering in Research Trends*, 11(12), pp.1-9.
- Norman, E.S., Brotherton, S.A. and Fried, R.T. (2008) *Work breakdown structures: The foundation for project management excellence*. 1st edn. Hoboken, NJ: John Wiley & Sons.
- Overgaard, S.M., Graham, M.G., Brereton, T., Pencina, M.J., Halamka, J.D., Vidal, D.E. and Economou-Zavlanos, N.J., 2023. Implementing quality management systems to close the AI translation gap and facilitate safe, ethical, and effective health AI solutions. *NPJ digital medicine*, 6(1), p.218.
- Rose, K., 2014. *Project quality management: why, what and how*. J. Ross Publishing.
- Schwalbe, K. (2018) *Information technology project management*. 9th edn. Boston, MA: Cengage Learning.

Shah, K.N., Gami, S.J. and Trehan, A., 2024. An intelligent approach to data quality management AI-Powered quality monitoring in analytics. *International Journal of Advanced Research in Science Communication and Technology*, 4(3), pp.109-119.

Țîrcovnicu, G.I. and Hațegan, C.D., 2023. Integration of artificial intelligence in the risk management process: An analysis of opportunities and challenges. *Journal of Financial Studies*, 8(15), pp.198-214.

Appendix

A1 Meeting Minutes

First meeting minute

Minutes (Week #4)	
Title	1 st GU's ICT Team Meeting
Attendees	Charles Lee, Lavinia Adams, Dr. Zhaohui Tang, Daniel Chavez, Paulina Casillas, Aswathy Sreekala, Faisal Khan
Apologies	
Location	Online via Teams
Agenda	1. Project objectives and expected outcomes 2. System architecture and flow 3. Proposed work plan 4. Q&A

Decisions and Memos	· Weekly meetings will be held to discuss project progress, current tasks, and upcoming work. · Open communication between the client and PDF Agents Team. · Weekly progress reports will be sent via email, and will include - o Summary of current work and completed tasks - o What are the next steps - o Any issues or challenges encountered - o A help section · Make sure that any communication that requires a response is sent with plenty of time, minimum 24 hrs. to provide feedback. · Show the client every time any progress is made, to keep them fully engaged in the decision-making progress. · The final prototype is required to meet with all the operational and security requirements, including: - o Operating entirely locally, no cloud services or external data exchange. - o Ability to read Word files, pdf and excel if possible but not required. - o No authentication required for login - o Designed for single computer deployment, and accessible to any user with access to a computer. · Source code management will be maintained using GitHub.
----------------------------	--

New Tasks Allocation in This Week			
Task	Description	Who	Due
1	Work on Assignment 1 – final project plan	PDF Agents	23 march
2	Review and improve project work plan	PDF Agents	23 march
3	Conduct additional research on proposed technologies and implementation approaches	PDF Agents	23 march
4	Begin development environment setup	PDF Agents	

Second Meeting Minutes

Minutes (Week #6)	
Title	2nd USQ team meeting
Attendees	Charles Lee, Lavinia Adams, Daniel Chavez, Paulina Casillas, Aswathy Sreekala, Faisal Khan
Apologies	Dr. Zhaohui Tang
Location	Online via Teams
Agenda	1. Review of progress and current tasks 2. Discuss completed tasks until this point 3. Outline next steps coming for upcoming weeks
Decisions and Memos	• Continue to update the Work Breakdown Structure as needed. ◦ Add new tasks if needed. ◦ The initial proposal serves as a baseline and does not need to be strictly followed; however, it should be updated to accurately reflect any changes in the project structure. • Share the GitHub repository with the AKA Studio team so they can review ongoing work. • Maintain weekly progress reports. ◦ If possible, include a brief snapshot of the WBS just to provide a clear overview of current progress. ◦ It is not necessary that all team members attend every meeting if their schedule doesn't allow it, as long as one team member attends. ◦ The attending member is responsible for taking notes and sharing the meeting minutes with absent team members • Continue sending minutes for every meeting.

Task Check for Last Week

Task	Criteria	Who	Due	Completed (Y/N)
1	Work on Assignment 1 – final project plan	PDF Agents	23 march	Y
2	Review and improve project work plan	PDF Agents	23 march	Y
3	Conduct additional research on proposed technologies and implementation approaches	PDF Agents	23 march	Y
4	Begin development environment setup	PDF Agents		Y

New Tasks Allocation			
Task	Description	Who	Due
1	Share the GitHub repository with Charles and Lavinia so that they can have a look.	Daniel Chavez	
2	Continue working on data ingestion, preparation and retrieval pipeline	PDF Agents	13/04

Third Meeting Minutes

Minutes (Week #8)	
Title	3rd USQ team meeting
Attendees	Charles Lee, Lavinia Adams, Daniel Chavez, Paulina Casillas, Aswathy Sreekala, Faisal Khan
Apologies	Dr. Zhaohui Tang
Location	Online via Teams
Agenda	1. Review of progress and current tasks 2. Discuss completed tasks until this point 3. Outline next steps coming for upcoming weeks

Decisions and Memos	• The system should be able to: ○ Search across all documents without requiring users to specify a file. ○ Only return information strictly from the documents (no hallucinations). ○ Clearly handle cases where a document is not found (no made-up answers). • Testing will be prioritized before adding advanced features such as graphs. ○ The current UI structure is sufficient for now, no major redesign is required at this stage. ○ Indexing strategy: ■ Use multiple indexes instead of overwriting. ■ Allow selecting indexes based on directories. ■ Provide an option to remove directories if limits are reached. ■ Show which documents are already indexed ○ Work on file version, the system must be able to identify differences and changes in different versions of a certain file. ■ Detect changes ■ Identify content differences accurately ■ Create multiple versions of existing files with slightly differences and test how the system reacts.
----------------------------	--

Task Check for Last Week				
Task	Criteria	Who	Due	Completed (Y/N)
1	Share the GitHub repository with Charles and Lavinia so that they can have a look.	Daniel Chavez		Y
2	Continue working on data ingestion, preparation and retrieval pipeline	PDF Agents	13/04	Y

New Tasks Allocation			
Task	Description	Who	Due

1	System testing with longer documents, including other type of data and considering all possible scenarios.	PDF Team	27/04
2	Retrieval improvements	PDF Team	27/04
3	Indexing implementation	PDF Team	27/04

Fourth Meeting Minutes

Minutes (Week #10)	
Title	4 th USQ team meeting
Attendees	Charles Lee, Daniel Chavez, Paulina Casillas, Aswathy Sreekala, Faisal Khan
Apologies	Dr. Zhaohui Tang, Lavinia Adams
Location	Online via Teams
Agenda	1. Review of progress and current tasks 2. Discuss completed tasks until this point 3. Outline next steps coming for upcoming weeks

Decisions and Memos	- Following the previous meeting, a request was made to enable the system to detect changes across different versions of a file. Due to current model limitations, this feature cannot be implemented in the existing prototype. - It was proposed that file version management could be developed as a separate tool or application. However, since the primary goal of the prototype is document summarisation and insight retrieval, this feature was deemed out of scope and will not be pursued due to time and scope constraints. - An index management functionality has been integrated into the prototype, allowing users to: - Use multiple indexes - Select a desired index - Remove directories - The team will begin testing the retrieval of information from Excel files to evaluate model performance. - The team agreed to prioritise insight retrieval improvements - A setup guide and user documentation will be prepare for handover.
----------------------------	--

Task Check for Last Week				
Task	Criteria	Who	Due	Completed (Y/N)
1	System testing with longer documents, including other type of data and considering all possible scenarios.	PDF TEAM	27/04	Y
2	Retrieval improvements	PDF TEAM	27/04	Y
3	Indexing implementation	PDF TEAM	27/04	Y

New Tasks Allocation			
Task	Description	Who	Due
1	Develop Excel file information retrieval	PDF Team	08 / 05

2	Prepare setup guide and user documentation	PDF Team	08/05
3	Create a video demonstration	PDF Team	08 / 05
4	Improve and debug the current prototype	PDF Team	08 / 05
5	Work on assessment 2	PDF Team	08 / 05

A2 Log Sheets

In this following link, the different log sheets from the team members are stored:

[📄 Log sheets Project INSIGHT.AI](#)

A3 Video presentation

The video presentation can be found in the following link:

[📄 VideoPresentation_INSIGHT.AI](#)